

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación gráfica, visión computacional y multimedia				
TÍTULO DE LA PRÁCTICA:	<i>Procesamiento de Imágenes</i>				
NÚMERO DE PRÁCTICA:	<i>07</i>	AÑO LECTIVO:	<i>2026A</i>	NRO. SEMESTRE:	<i>IX</i>
FECHA DE PRESENTACIÓN	<i>11/06/2026</i>	HORA DE PRESENTACIÓN			
INTEGRANTE (s): Maxs Sebastian Joaquin Forocca Mamani				NOTA:	
DOCENTE(s): <i>RENE ALONSO NIETO VALENCIA</i>					

SOLUCIÓN Y RESULTADOS	
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS	
1. Introducción y Objetivos	
1.1. Introduccion	El procesamiento de imágenes digitales constituye uno de los pilares fundamentales de la visión computacional y las aplicaciones multimedia modernas. En esencia, una imagen digital no es más que una matriz bidimensional de datos donde cada elemento (píxel) representa un valor de intensidad lumínica o de color. El análisis y la manipulación de estas estructuras permiten optimizar la información visual, eliminar ruido , resaltar características críticas (como bordes o formas) y segmentar regiones de interés para sistemas automatizados. A través del uso de librerías especializadas como OpenCV y bibliotecas de cálculo numérico como NumPy, es posible descomponer la información cromática, reestructurar dimensiones matriciales y generar entornos interactivos capaces de reaccionar a periféricos de entrada. Este laboratorio aborda de forma práctica la manipulación directa de los canales de color, transformaciones lógicas puntuales, segmentación por umbrales y la gestión de eventos síncronos en entornos de interfaz gráfica de usuario (GUI).
1.2. Objetivos	

1.2.1. Objetivo General

Desarrollar e implementar algoritmos fundamentales de procesamiento de imágenes digitales y diseño de herramientas interactivas utilizando la biblioteca OpenCV en el lenguaje de programación Python.

1.2.2. Objetivos Específicos

1. Manipular de forma matricial las dimensiones de múltiples imágenes concurrentes para homogeneizar sus geometrías.
2. Comprender la disposición de los canales de color en el formato BGR (Blue, Green, Red) y aplicar fusiones selectivas entre capas de distintas fuentes.
3. Aplicar transformaciones puntuales lineales y no lineales, tales como la inversión de bits (negativo) y la conversión a escala de grises.
4. Diseñar interfaces gráficas dinámicas que respondan de manera eficiente a eventos asíncronos del mouse y del teclado para tareas de visualización y dibujo computarizado.
5. Implementar técnicas de binarización por umbral (thresholding) para aislar rangos de intensidades lumínicas específicos en una escena.

2. DESARROLLO DE LOS EJERCICIOS

2.1. Redimensionar imágenes

Código Fuente:

Python

```
import cv2
import os

# Crear carpeta de resultados si no existe
os.makedirs("resultados", exist_ok=True)

# 1. Cargar las 3 imágenes
img1 = cv2.imread("imagenes_origen/mrincreible.jpg")
img2 = cv2.imread("imagenes_origen/perrito.jpg")
img3 = cv2.imread("imagenes_origen/roni.jpg")

# 2. Encontrar las dimensiones máximas (alto, ancho)
# img.shape retorna (alto, ancho, canales)
max_alto = max(img1.shape[0], img2.shape[0], img3.shape[0])
max_ancho = max(img1.shape[1], img2.shape[1], img3.shape[1])
dimension_final = (max_ancho, max_alto) # OpenCV usa (ancho, alto) para resize

# 3. Redimensionar todas a la dimensión más grande
```

```
img1_res = cv2.resize(img1, dimension_final, interpolation=cv2.INTER_CUBIC)
img2_res = cv2.resize(img2, dimension_final, interpolation=cv2.INTER_CUBIC)
img3_res = cv2.resize(img3, dimension_final, interpolation=cv2.INTER_CUBIC)
```

4. Guardar las imágenes temporales redimensionadas para el siguiente paso

```
cv2.imwrite("resultados/img1_res.png", img1_res)
cv2.imwrite("resultados/img2_res.png", img2_res)
cv2.imwrite("resultados/img3_res.png", img3_res)
```

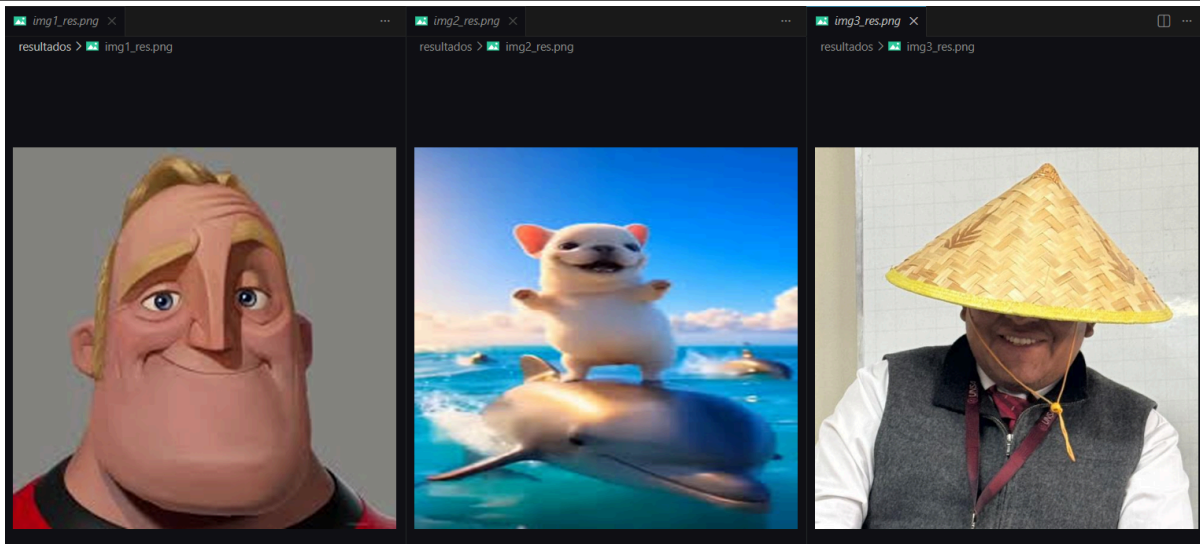
```
print(f"Imágenes redimensionadas con éxito a: {dimension_final}")
```

Explicación del Funcionamiento:

El script inicializa automatizando la creación del directorio de almacenamiento mediante `os.makedirs`. Posteriormente, carga tres imágenes en memoria a través de `cv2.imread`. La lógica central del programa radica en la flexibilidad geométrica: en lugar de utilizar dimensiones fijas estáticas, se leen dinámicamente las propiedades de las matrices de las imágenes usando el atributo `.shape` (el cual retorna una tupla con la estructura [alto, ancho, canales]).

Mediante la función nativa `max()`, el algoritmo determina de forma comparativa cuál es el valor de altura (`max_alto`) y ancho (`max_ancho`) más elevado entre las tres entradas. Con estos valores máximos consolida la variable `dimension_final`. Finalmente, se ejecuta la función `cv2.resize` sobre cada imagen empleando la interpolación cúbica `cv2.INTER_CUBIC`, una técnica de remuestreo basada en los 4x4 píxeles vecinos que, si bien requiere mayor costo computacional, preserva una calidad visual óptima y suavizada ante reescalados hacia dimensiones superiores. Las matrices resultantes se exportan a disco en formato `.png` para mitigar la pérdida de información por compresión.

Resultados Obtenidos:



2.2. Crear nueva imagen con canales de color combinados

Código Fuente:

Python

```
import cv2
```

```
# Cargar las imágenes ya redimensionadas del paso anterior
```

```
img1 = cv2.imread("resultados/img1_res.png")
```

```
img2 = cv2.imread("resultados/img2_res.png")
```

```
img3 = cv2.imread("resultados/img3_res.png")
```

```
# Separar los canales B, G, R de cada imagen independiente
```

```
# Recuerda: OpenCV extrae en orden B=0, G=1, R=2
```

```
_, _, r_img1 = cv2.split(img1)
```

```
_, g_img2, _ = cv2.split(img2)
```

```
b_img3, _, _ = cv2.split(img3)
```

```
# Fusionar siguiendo estrictamente el formato BGR de OpenCV
```

```
imagen_combinada = cv2.merge([b_img3, g_img2, r_img1])
```

```
# Guardar y mostrar el resultado
```

```
cv2.imwrite("resultados/imagen_combinada.png", imagen_combinada)
```

```
cv2.imshow("Imagen Combinada", imagen_combinada)
```

```
cv2.waitKey(0)
```

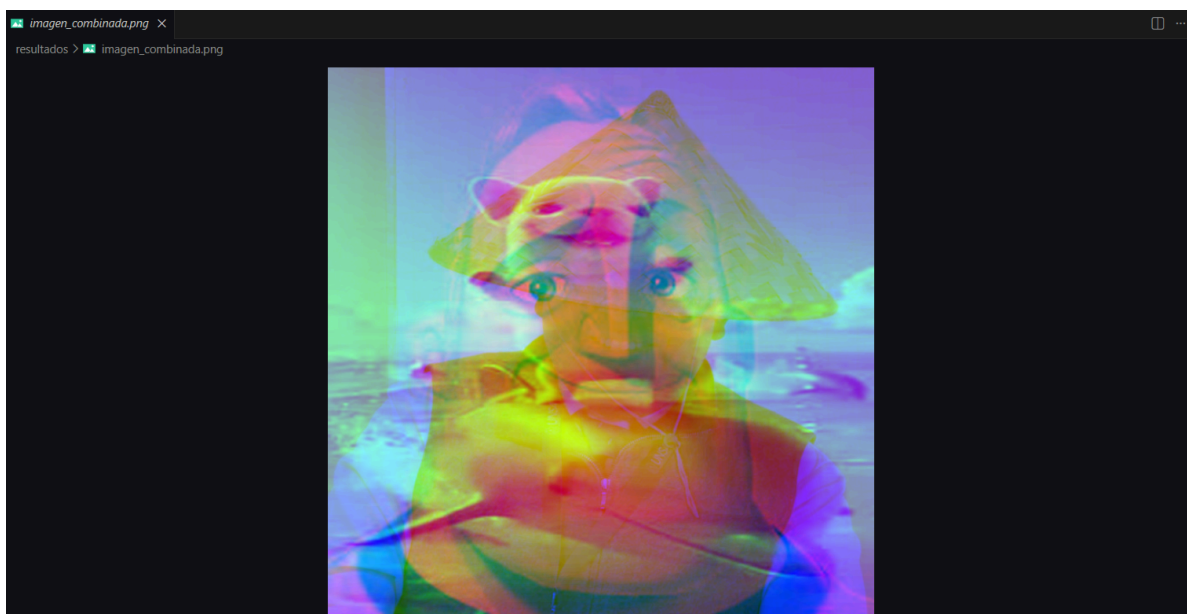
```
cv2.destroyAllWindows()
```

Explicación del Funcionamiento:

Esta sección recupera las tres imágenes previamente redimensionadas, asegurando que sus dimensiones de matriz sean idénticas y posibiliten operaciones algebraicas uniformes. En OpenCV, la representación por defecto de las imágenes a color no sigue el estándar convencional RGB, sino el orden inverso BGR (Blue, Green, Red).

Utilizando la función `cv2.split()`, el script descompone cada una de las imágenes tridimensionales en sus tres capas o canales bidimensionales independientes. El diseño del programa extrae de forma quirúrgica los canales solicitados por la guía: el canal rojo de la primera imagen (`r_img1`), el canal verde de la segunda (`g_img2`) y el canal azul de la tercera (`b_img3`). Tras aislar estas matrices de un solo canal, se invoca la función `cv2.merge()`. Al pasarle como argumento la lista ordenada [`b_img3`, `g_img2`, `r_img1`], OpenCV fusiona los planos individuales en una sola matriz tridimensional compacta, dando origen a una composición cromática híbrida inédita que es almacenada en el disco duro.

Resultados Obtenidos:



2.3. Conversión a negativo y a escala de grises Código Fuente:

```
Python
import cv2
```

```
# 1. Cargar la imagen combinada
```

```
img_comb = cv2.imread("resultados/imagen_combinada.png")
```

```
# 2. Calcular el negativo (invertir bits: 255 - valor_pixel)
```

```
img_negativo = cv2.bitwise_not(img_comb)
```

```
cv2.imwrite("resultados/imagen_negativa.png", img_negativo)
```

```
# 3. Cargar la imagen en escala de grises directamente desde el archivo guardado
```

```
# El parámetro cv2.IMREAD_GRAYSCALE (o el número 0) hace la conversión al leer
```

```
img_grises = cv2.imread("resultados/imagen_negativa.png", cv2.IMREAD_GRAYSCALE)
```

```
cv2.imwrite("resultados/imagen_grises.png", img_grises)
```

```
# Mostrar ambos resultados
```

```
cv2.imshow("Negativo", img_negativo)
```

```
cv2.imshow("Escala de Grises", img_grises)
```

```
cv2.waitKey(0)
```

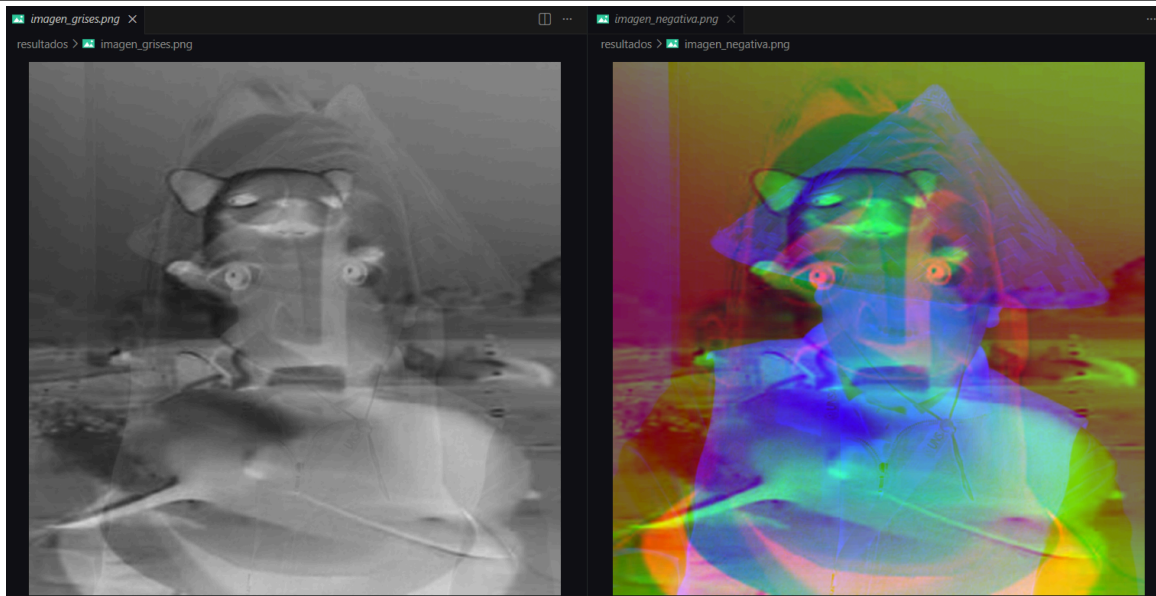
```
cv2.destroyAllWindows()
```

Explicación del Funcionamiento:

El programa carga la imagen compuesta en el paso anterior y ejecuta una transformación de inversión de color lineal. En un espacio de color de 8 bits por canal, los valores numéricos de los píxeles oscilan estrictamente entre \$0\$ (ausencia de color) y \$255\$ (saturación total). Para obtener el negativo, se aplica la función optimizada `cv2.bitwise_not()`, la cual realiza una operación lógica NOT a nivel de bits sobre la matriz, lo que matemáticamente equivale a restar el valor actual del píxel al límite superior del espacio de color ($255 - \text{valor_píxel}$).

Inmediatamente después, el script guarda dicha salida y procede a realizar la conversión a escala de grises. Para optimizar líneas de ejecución, en lugar de invocar funciones de conversión de color en caliente (como `cv2.cvtColor`), se realiza una lectura directa del archivo negativo recién guardado pasando como segundo argumento el flag `cv2.IMREAD_GRAYSCALE` (equivalente numérico a 0). Esto le indica al decodificador de OpenCV que procese el archivo aplicando directamente una ponderación de luminancia estándar del ojo humano a los canales, reduciendo la imagen a una sola matriz de intensidades de un canal, la cual es guardada bajo el nombre de `imagen_grises.png`.

Resultados Obtenidos:



2.4. Aplicación interactiva de visualización de canales de color

Código Fuente:

Python

```
import cv2
import numpy as np

img = cv2.imread("resultados/imagen_combinada.png")
b_org, g_org, r_org = cv2.split(img)

# Variables de estado booleanas para controlar la visibilidad
ver_r, ver_g, ver_b = True, True, True

print("Controles:\n'r' -> Alternar Rojo\n'g' -> Alternar Verde\n'b' -> Alternar
Azul\n'ESC' -> Salir")

while True:
    # Si el canal está desactivado, creamos una matriz vacía (ceros) del mismo tamaño
    r = r_org if ver_r else np.zeros_like(r_org)
    g = g_org if ver_g else np.zeros_like(g_org)
    b = b_org if ver_b else np.zeros_like(b_org)

    # Re-acoplar la imagen dinámica
    img_dinamica = cv2.merge([b, g, r])
    cv2.imshow("Visor Interactivo de Canales", img_dinamica)
```



```
# Capturar la tecla presionada (espera 10ms)
tecla = cv2.waitKey(10) & 0xFF

if tecla == ord('r') or tecla == ord('R'):
    ver_r = not ver_r
elif tecla == ord('g') or tecla == ord('G'):
    ver_g = not ver_g
elif tecla == ord('b') or tecla == ord('B'):
    ver_b = not ver_b
elif tecla == 27: # Código ASCII de la tecla Escape
    break

cv2.destroyAllWindows()
```

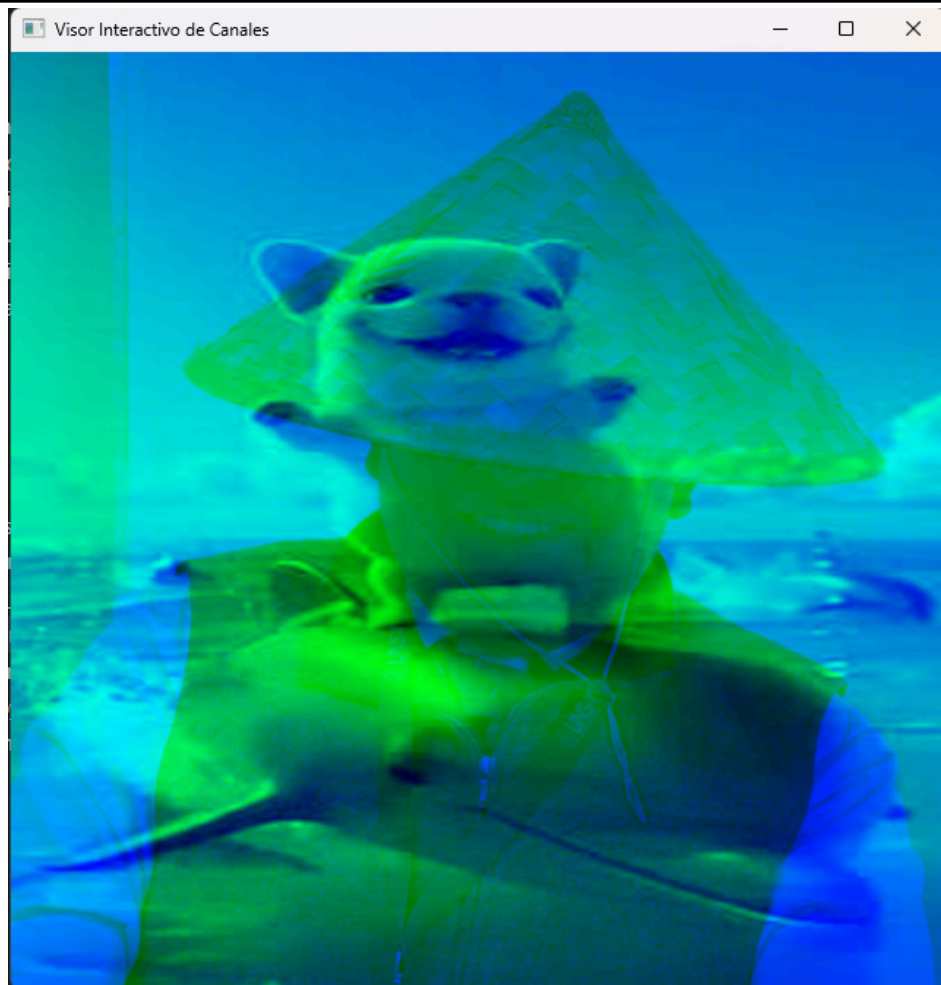
Explicación del Funcionamiento:

El script implementa un visor dinámico de control de estados empleando un ciclo infinito `while True`. En primera instancia, carga la imagen combinada y la segmenta en sus canales fundamentales (`b_org`, `g_org`, `r_org`). Se definen tres banderas booleanas (`ver_r`, `ver_g`, `ver_b`) inicializadas en valor verdadero (`True`), encargadas de rastrear qué componentes de color deben ser renderizados en pantalla.

Dentro del ciclo, se evalúa el estado de cada bandera mediante el operador condicional ternario en Python: si la bandera es verdadera, el canal conserva sus valores de intensidad originales; si es falsa, se genera instantáneamente una matriz de ceros (color negro absoluto) con las mismas dimensiones físicas empleando la función de alta eficiencia de NumPy `np.zeros_like()`. Posterior a esta validación, los tres canales (activos o nulos) se recombinan dinámicamente con `cv2.merge()` y se despliegan en la ventana mediante `cv2.imshow`.

Para la captura de las directivas del usuario, se utiliza `cv2.waitKey(10) & 0xFF`. El retraso de 10 milisegundos mantiene el ciclo fluido sin saturar la CPU. Si se detecta la pulsación de caracteres específicos ('r', 'g', 'b' en mayúsculas o minúsculas), se ejecuta una operación de negación booleana (`not`) sobre la respectiva bandera de control, alterando el flujo de la siguiente iteración. El bucle se rompe de forma segura al recibir el código ASCII 27, correspondiente a la tecla de Escape (ESC).

Resultados Obtenidos:



2.5. Dibujo de figuras y texto en imágenes

Código Fuente:

Python

```
import cv2
```

```
# Cargar una de las imágenes originales (asegúrate de que tenga una persona/animal)  
img = cv2.imread("imagenes_origen/mrincreible.jpg")
```

```
# Definir coordenadas manuales estimadas de la cara (ajusta x, y según imagen de  
mrincreible)
```

```
centro_x, centro_y = 112, 112
```

```
radio = 80
```

```
# 1. Dibujar el círculo (Imagen, Centro, Radio, Color BGR, Grosor de línea)
```

```
cv2.circle(img, (centro_x, centro_y), radio, (0, 255, 0), 3)
```

```
# 2. Añadir texto (Imagen, Texto, Origen_Coordenadas, Fuente, Escala, Color BGR, Grosor)
```

```
posicion_texto = (centro_x - radio, centro_y - radio - 15)
```

```
cv2.putText(img, "Mr Increible", posicion_texto,  
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2, cv2.LINE_AA)
```

```
# Guardar resultado
```

```
cv2.imwrite("resultados/imagen_anotada.png", img)
```

```
cv2.imshow("Anotaciones", img)
```

```
cv2.waitKey(0)
```

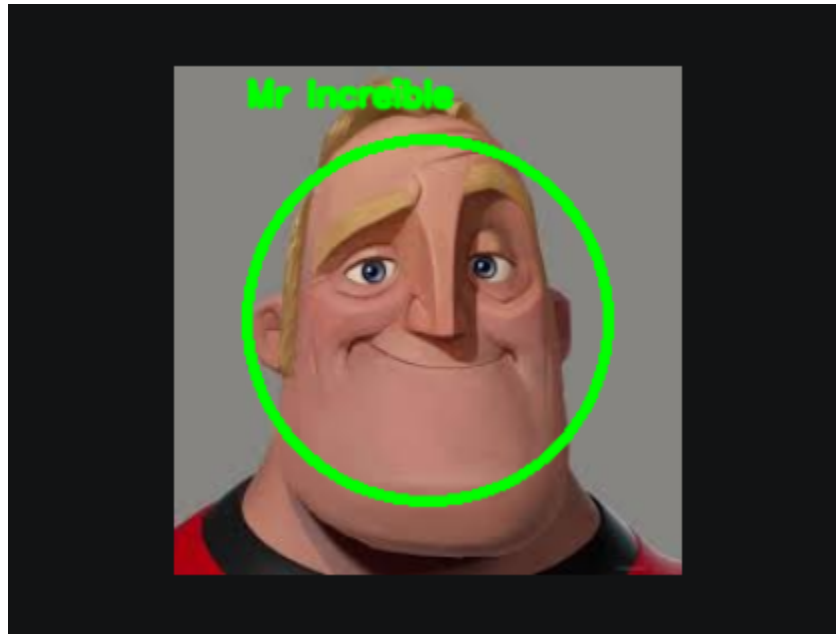
```
cv2.destroyAllWindows()
```

Explicación del Funcionamiento:

Este módulo se enfoca en la superposición de primitivas gráficas vectoriales sobre imágenes rasterizadas. Se efectúa la lectura de una imagen de origen que contiene un sujeto (en este caso particular, el personaje "Mr Increible"). Mediante coordenadas de píxeles previamente estimadas de forma manual en el espacio bidimensional de la imagen, se establecen el centro focal de la cara (`centro_x`, `centro_y`) y la longitud del radio del vector de enmarcado.

La primera operación gráfica utiliza la función `cv2.circle()`, la cual modifica de forma destructiva los píxeles de la matriz original trazando una circunferencia con el color configurado en formato BGR (en este caso (0, 255, 0), que representa verde puro) y un grosor de trazo de 3 píxeles. Posteriormente, se define una posición relativa superior para el texto con el fin de evitar colisiones visuales con el círculo. La inserción tipográfica se gestiona con `cv2.putText()`, empleando la fuente vectorial estandarizada `cv2.FONT_HERSHEY_SIMPLEX` a una escala de 0.5. El parámetro final `cv2.LINE_AA` activa un algoritmo de antialiasing (antialiado por líneas de conexión), el cual suaviza los bordes curvos de las letras aplicando una degradación de color en los píxeles limítrofes, evitando el efecto de pixelado "serruchado".

Resultados Obtenidos:



2.6. Aplicación de umbral binario

Código Fuente:

Python

```
import cv2

# El umbral binario requiere obligatoriamente una imagen en escala de grises
img_gris = cv2.imread("resultados/imagen_grises.png", cv2.IMREAD_GRAYSCALE)

# Aplicar umbral binario: si el píxel supera 127 se vuelve 255 (blanco), sino 0 (negro)
valor_umbral = 127
max_valor = 255
_, img_threshold = cv2.threshold(img_gris, valor_umbral, max_valor, cv2.THRESH_BINARY)

# Guardar y mostrar
cv2.imwrite("resultados/imagen_umbral.png", img_threshold)
cv2.imshow("Umbral Binario", img_threshold)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explicación del Funcionamiento:

El procesamiento de binarización por umbral se ejecuta sobre la imagen previamente guardada en escala de grises, cumpliendo con la restricción matemática del algoritmo, la

cual requiere operar sobre un único canal de intensidades lumínicas. El script define un valor_umbral crítico de 127 y un max_valor fijado en 255.

La segmentación se realiza invocando la función `cv2.threshold()`, enviando como parámetro de tipo de umbralización el flag `cv2.THRESH_BINARY`. El funcionamiento del algoritmo interno evalúa matemáticamente cada píxel de la matriz de entrada de acuerdo con la siguiente función condicional:

$$\text{dst}(x, y) = \begin{cases} \text{max_valor} & \text{si } \text{src}(x, y) > \text{valor_umbral} \\ 0 & \text{en caso contrario} \end{cases}$$

Resultados Obtenidos:



2.7. Programa de dibujo interactivo con eventos de mouse y teclado

Código Fuente:

```
Python  
import cv2
```

```
import numpy as np
import os

# Configuración inicial del lienzo blanco
lienzo = np.ones((600, 800, 3), dtype="uint8") * 255
historial = [lienzo.copy()] # Historial para la función deshacer

# Variables globales de control
dibujando = False
modo_figura = 'c' # 'c' para círculo, 'r' para rectángulo
ix, iy = -1, -1
cx, cy = -1, -1 # Coordenadas actuales del mouse mientras se arrastra

print("Controles del lienzo:")
print("'c' -> Modo Círculo\n'r' -> Modo Rectángulo\n'z' -> Deshacer\n's' -> Guardar Dibujo\n'ESC' -> Salir\n" + "-"*30)

def dibujar(event, x, y, flags, param):
    global ix, iy, cx, cy, dibujando, lienzo, historial, modo_figura

    if event == cv2.EVENT_LBUTTONDOWN:
        dibujando = True
        ix, iy = x, y
        cx, cy = x, y # Inicializar la posición actual

    elif event == cv2.EVENT_MOUSEMOVE:
        if dibujando:
            cx, cy = x, y # Solo actualizamos las coordenadas actuales

    elif event == cv2.EVENT_LBUTTONUP:
        dibujando = False
        # Guardamos el estado actual en el historial ANTES de modificar el lienzo
        historial.append(lienzo.copy())

        # Consolidar el dibujo definitivo en el lienzo real
        if modo_figura == 'r':
            cv2.rectangle(lienzo, (ix, iy), (x, y), (255, 0, 0), 2)
        elif modo_figura == 'c':
            radio = int(((x - ix)**2 + (y - iy)**2)**0.5)
            cv2.circle(lienzo, (ix, iy), radio, (0, 0, 255), 2)

# Vincular la ventana con la función del mouse
cv2.namedWindow("Pizarra Interactiva")
cv2.setMouseCallback("Pizarra Interactiva", dibujar)
```

```
while True:
    # Si estamos arrastrando el mouse, mostramos la previsualización en tiempo real
    if dibujando:
        copia_temporal = lienzo.copy()
        if modo_figura == 'r':
            cv2.rectangle(copia_temporal, (ix, iy), (cx, cy), (255, 0, 0), 2)
        elif modo_figura == 'c':
            radio = int(((cx - ix)**2 + (cy - iy)**2)**0.5)
            cv2.circle(copia_temporal, (ix, iy), radio, (0, 0, 255), 2)

        cv2.imshow("Pizarra Interactiva", copia_temporal)
    else:
        # Si no se está dibujando, se muestra el lienzo definitivo estable
        cv2.imshow("Pizarra Interactiva", lienzo)

    tecla = cv2.waitKey(1) & 0xFF

    if tecla == ord('r'):
        modo_figura = 'r'
        print("Modo cambiado a: Rectángulo")
    elif tecla == ord('c'):
        modo_figura = 'c'
        print("Modo cambiado a: Círculo")
    elif tecla == ord('z'):
        if len(historial) > 0:
            lienzo = historial.pop() # Recupera el último estado guardado
            print("Deshacer aplicado.")
        else:
            print("No hay más acciones para deshacer.")
    elif tecla == ord('s'):
        # Validar existencia de la carpeta para evitar crashes
        if not os.path.exists("resultados"):
            os.makedirs("resultados")
        cv2.imwrite("resultados/dibujo_final.png", lienzo)
        print("Dibujo guardado exitosamente en 'resultados/dibujo_final.png'.")
    elif tecla == 27: # ESC
        break

cv2.destroyAllWindows()
```

Explicación del Funcionamiento:

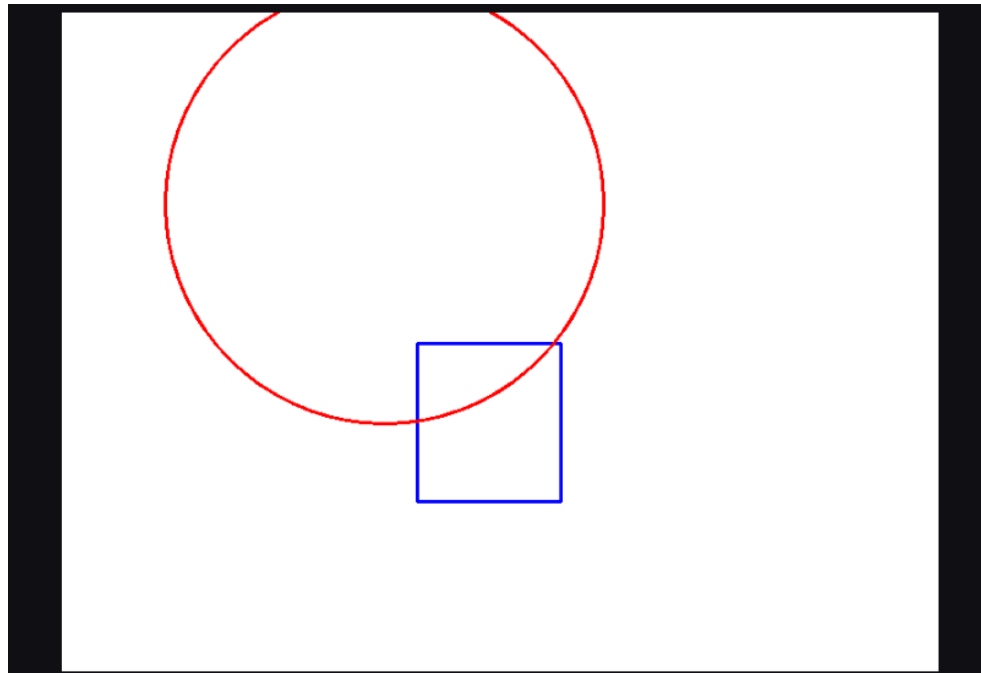
Este script representa una solución avanzada de software que integra programación orientada a eventos y un patrón básico de gestión de estados (Undo/Deshacer). Comienza creando un lienzo sintético bidimensional utilizando NumPy: `np.ones((600,`

800, 3), dtype="uint8") * 255 genera una matriz de 3 canales completamente inicializada con el valor numérico 255, resultando en un fondo blanco puro. Se define una lista llamada historial que almacena copias profundas de la matriz del lienzo (lienzo.copy()), actuando como una pila (stack) de memoria para revertir cambios. La interactividad con el puntero se gestiona mediante la función de callback dibujar(), la cual está vinculada a la ventana gráfica a través de cv2.setMouseCallback(). Esta función intercepta los eventos del sistema de la siguiente manera:

- cv2.EVENT_LBUTTONDOWN: Activa la bandera dibujando, congela las coordenadas iniciales del clic en (ix, iy) y asigna los valores de arrastre actual (cx, cy).
- cv2.EVENT_MOUSEMOVE: Si la bandera dibujando se encuentra activa, actualiza de forma continua las coordenadas dinámicas del puntero en las variables globales (cx, cy).
- cv2.EVENT_LBUTTONUP: Desactiva la bandera dibujando, realiza un respaldo del estado actual del lienzo insertándolo en la lista historial mediante .append(), y ejecuta el dibujo vectorial definitivo en el lienzo principal de acuerdo al estado de modo_figura ('r' para rectángulo con cv2.rectangle o 'c' para círculo calculando el radio por el teorema de Pitágoras).

El bucle principal regula el refresco de la pantalla a una tasa extremadamente alta (cv2.waitKey(1)). La innovación técnica clave radica en la lógica de renderizado: mientras el usuario mantiene presionado el botón izquierdo y arrastra el mouse (dibujando == True), el programa genera una copia temporal volátil del lienzo estable (lienzo.copy()), dibuja la figura geométrica proyectada sobre dicha copia y la muestra. Esto genera un efecto visual de "previsualización elástica en tiempo real" sin alterar o corromper el lienzo definitivo. Las interrupciones de teclado capturan comandos en caliente para conmutar variables de modo visual, extraer y restaurar matrices de la pila historial con el método .pop() ante la tecla 'z', o invocar directivas de escritura en disco ante la tecla 's'.

Resultados Obtenidos:

**3. Enlace al Video** zuh-tytk-hba (2026-06-12 02:31 GMT-5)**4. Enlace a github: <https://github.com/MaxsForocca/LabComputacionGrafica>****II. SOLUCIÓN DEL CUESTIONARIO****1. ¿Qué otros modelos de colores existen para las imágenes?**

Más allá de los modelos matriciales basados en componentes primarias aditivas como RGB/BGR, el procesamiento de imágenes digitales y la visión artificial emplean diversos modelos matemáticos de color optimizados para tareas específicas:

- HSV / HSL (Hue, Saturation, Value / Lightness): Este modelo modela el color de manera más cercana a la percepción cognitiva humana. Descompone la imagen en Matiz (Tinte/Color), Saturación (Pureza del color) y Valor/Brillo. Es el modelo preferido en visión computacional para la segmentación de objetos basados en su color exacto, ya que separa por completo la información del color de las variaciones de iluminación ambiental.

- CMYK (Cyan, Magenta, Yellow, Key/Black): Un modelo de carácter sustractivo utilizado de forma exclusiva en la industria de la impresión física y dispositivos de salida tipográfica. Se fundamenta en la absorción de la luz por la combinación de pigmentos.
- YCrCb / YUV: Utilizado ampliamente en estándares de transmisión de video y compresión de archivos digitales (como las bases del formato JPEG). Separa la información en un componente de Luminancia (Y, detalles de brillo en blanco y negro) y dos componentes de Crominancia (Cr y Cb, diferencias de color). Permite aplicar submuestreo de crominancia, reduciendo el tamaño del archivo sin que el ojo humano note una pérdida de calidad apreciable.
- CIELAB ($L^*a^*b^*$): Un modelo de color con un espacio perceptual completamente uniforme especificado por la Comisión Internacional de la Iluminación. El canal L^* representa la luminosidad, a^* define la transición entre verde y rojo, y b^* define la transición entre azul y amarillo. Su ventaja matemática radica en que la distancia euclidiana entre dos puntos de color en este espacio coincide proporcionalmente con la diferencia visual percibida por el ojo humano, siendo crucial para el control de calidad industrial del color.

2. ¿Qué otros cambios se pueden realizar a las imágenes con OpenCV?

OpenCV es una librería masiva de grado industrial que permite realizar una gama casi ilimitada de transformaciones sobre las matrices de píxeles, subdivididas principalmente en:

- Transformaciones Geométricas y Afines: Rotaciones espaciales, traslaciones sobre los ejes cartesianos, escalados asimétricos, cizallamiento (shearing) y transformaciones de perspectiva no afines (homografías), útiles para enderezar documentos u objetos capturados desde ángulos oblicuos.
- Filtrado Digital y Difuminado: Operaciones de convolución espacial por medio de matrices de Kernel para el suavizado de imágenes y reducción de ruido, tales como el Filtro Gaussiano, Filtro de Mediana (altamente efectivo contra el ruido de tipo "sal y pimienta") y Filtros Bilaterales que reducen el ruido preservando la nitidez de los bordes.
- Operaciones Morfológicas: Aplicadas generalmente sobre estructuras binarias. Operaciones como la Erosión (reduce el tamaño de estructuras blancas), Dilatación (expande las estructuras), Opening (elimina ruido aislado exterior) y Closing (cierra huecos o grietas internas en objetos segmentados).
- Extracción de Características y Bordes: Algoritmos avanzados de gradientes espaciales como los operadores Sobel, Scharr y Laplaciano, así como el sintonizador Canny para la detección precisa de contornos y bordes vectorizables.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

3. ¿Qué otros cambios se pueden realizar a las imágenes con OpenCV?

El umbralizado binario (Binary Thresholding) constituye la técnica de segmentación más simple, rápida y eficiente en términos de procesamiento de datos. Su uso principal radica en actuar como una etapa de preprocesamiento crítico para transformar imágenes complejas de escala de grises en mapas binarios de máscaras lógicas. Entre sus aplicaciones industriales más comunes se encuentran:

- Segmentación de Objetos y Aislamiento de Regiones de Interés (ROI): Permite separar de forma inmediata elementos que presenten un contraste marcado respecto a su fondo. Por ejemplo, el conteo automatizado de células o bacterias en placas de Petri dentro de sistemas de bioingeniería, o la identificación de piezas metálicas sobre fajas transportadoras oscuras en entornos de manufactura automatizada.
- Preparación de Texto para Reconocimiento Óptico de Caracteres (OCR): Los motores de OCR (como Tesseract) requieren de forma mandatoria que los caracteres de un documento escaneado presenten un contraste absoluto (texto negro puro sobre fondo blanco puro). El umbral binario elimina sombras de escaneo, arrugas del papel o imperfecciones del color de fondo, maximizando drásticamente la precisión del reconocimiento de las letras.
- Generación de Máscaras Bitwise: Permite aislar formas específicas para posteriormente utilizarlas como máscaras de bits lógicas (operaciones AND, OR). Con esto, se pueden recortar regiones exactas de una imagen a color original o superponer elementos gráficos sin alterar las zonas circundantes.
- Análisis de Formas e Inspección de Defectos: Facilita el cálculo posterior de propiedades geométricas como el área del objeto (conteo de píxeles activos \$255\$), perímetro, centroides y orientación de contornos, agilizando la detección de anomalías estructurales en sistemas de control de calidad visual.

III. CONCLUSIONES

Se concluye que el procesamiento de imágenes en el dominio espacial mediante la manipulación directa de matrices numéricas (NumPy) provee un control de grano fino y de alta eficiencia sobre los componentes cromáticos de una escena, demostrando que operaciones complejas de color pueden reducirse a sutiles reestructuraciones algebraicas y condicionales de píxeles.

Se corrobora que la arquitectura del espacio de color por defecto en OpenCV (ordenamiento BGR) requiere un tratamiento riguroso y consciente al momento de interactuar con otras librerías de visualización o al realizar

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

fusiones de canales independientes, evitando distorsiones cromáticas involuntarias en las salidas definitivas del software.

Se evidencia que el algoritmo de umbralización binaria actúa como un filtro de segmentación fundamental no lineal sumamente óptimo, cuya efectividad técnica es estrictamente dependiente de una correcta homogeneidad lumínica previa y de una adecuada selección del valor crítico de corte en el histograma de la escala de grises.

Se demuestra que el paradigma de programación orientada a eventos, integrada de forma nativa en los controladores de interfaces de OpenCV, permite construir soluciones interactivas en tiempo real capaces de gestionar flujos asíncronos de datos periféricos (mouse y teclado) sin comprometer la estabilidad ni la tasa de refresco visual del lienzo principal.

TROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] <https://github.com/opencv/opencv>
- [2] <https://www.geeksforgeeks.org/python/opencv-python-tutorial/>